Table des matières

Labo 04 OLAP — Fonctions de fenêtrage sur l'entrepôt 1
Partie 1 — Voir la différence : GROUP BY contre fenêtre 1
Partie 2 — Les grandes familles 2
Et ensuite ? 4

Labo 04 OLAP — Fonctions de fenêtrage sur l'entrepôt

Laboratoire du Cours 04, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

On met en pratique les fonctions de fenêtrage sur l'entrepôt AdventureWorks DW, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l'entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Comptez une demi-heure.

Les deux voies — Ce labo a un jumeau, `Labo_04_OLTP_SQL.md`, qui pose exactement les mêmes questions sur le schéma normalisé d'AdventureWorks dans PostgreSQL. Faites celui-ci d'abord, puis l'autre, et gardez-les ouverts côte à côte : les écarts que vous verrez sont la vraie leçon du cours.

Tapez chaque requête, lisez la colonne ajoutée, puis faites les petites variations demandées : c'est là que le déclic se produit.

Partie 1 — Voir la différence : `GROUP BY` contre fenêtre

Théorie : `Theorie_04_SQL.md`, sections 1 et 2.

D'abord le `GROUP BY`, un total par produit, une ligne par produit :

```
SELECT ProductKey, SUM(SalesAmount) AS total_produit
FROM dbo.FactInternetSales
GROUP BY ProductKey
ORDER BY total_produit DESC;
```

Maintenant la fenêtre : le même total, mais chaque vente reste visible.

```
SELECT SalesOrderNumber, ProductKey, SalesAmount,
       SUM(SalesAmount) OVER (PARTITION BY ProductKey) AS total_produit
FROM dbo.FactInternetSales
ORDER BY ProductKey;
```

Observez. Le premier résultat a une ligne par produit ; le second garde toutes les ventes et répète le total en face de chacune. Même calcul, deux usages.

© 2026 iAlexMG. Tous droits réservés.

Partie 2 — Les grandes familles

2.1 — Classer : ROW_NUMBER, RANK, DENSE_RANK

Théorie : `Theorie_04_SQL.md`, section 3.

Objectif : classer les produits du plus cher au moins cher, et voir comment les égalités sont traitées.

```sql
SELECT EnglishProductName, ListPrice,
       ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS no_ligne,
       RANK()       OVER (ORDER BY ListPrice DESC) AS rang,
       DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS rang_dense
FROM dbo.DimProduct
WHERE ListPrice > 0
ORDER BY ListPrice DESC;
```

Observez. Là où plusieurs produits ont le même prix, `ROW_NUMBER` donne des numéros tous différents, `RANK` répète le rang puis saute, `DENSE_RANK` répète sans sauter.

2.2 — Top N par groupe avec QUALIFY

Théorie : `Theorie_04_SQL.md`, sections 3 et 7.

Objectif : les 3 produits les plus chers de chaque sous-catégorie, en une seule requête.

```sql
SELECT ProductSubcategoryKey, EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE ListPrice > 0 AND ProductSubcategoryKey IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY ProductSubcategoryKey
                           ORDER BY ListPrice DESC, ProductKey) <= 3
ORDER BY ProductSubcategoryKey, ListPrice DESC;
```

Attendu : 92 lignes.

Observez. Le classement repart à 1 dans chaque sous-catégorie, et le `QUALIFY` ne garde que les trois premiers de chacune. Changez `<= 3` en `= 1` : vous n'avez plus que le champion de chaque sous-catégorie.

Notez le `ProductKey` en second critère du `ORDER BY` de la fenêtre. Il ne sert pas à trier, il sert à départager : plusieurs produits partagent le même prix, et sans lui le moteur choisirait arbitrairement lequel occupe la troisième place. Le résultat serait juste au sens du compte et instable d'une exécution à l'autre, ce qui est la pire combinaison pour un rapport qu'on relance chaque semaine. Un top N sans colonne de départage est un top N dont personne ne peut garantir le contenu.

© 2026 iAlexMG. Tous droits réservés.

Le labo transactionnel pose la même question et rend 93 lignes, avec des noms tous distincts. L'écart n'est pas du dialecte : `dbo.DimProduct` porte 606 lignes pour 504 noms de produits, parce qu'un produit dont le prix change y reçoit une nouvelle ligne. « HL Road Handlebars » sort deux fois du top 3, « AWC Logo Cap » trois fois : ce sont trois versions du même article. Une dimension à historique classe des versions, pas des entités, et il faut le savoir avant de présenter un top N à qui que ce soit. Le Cours 05 y revient avec `DimEmployee`.

2.3 — Regarder le mois précédent : `LAG`

Théorie : `Theorie_04_SQL.md`, section 4.

Objectif : les ventes par mois, avec la variation par rapport au mois précédent.

```sql
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois)          AS ventes_mois_prec,
       ventes - LAG(ventes) OVER (ORDER BY annee, mois) AS variation
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
) AS mensuel
ORDER BY annee, mois;
```

Observez. La première ligne a `ventes_mois_prec` à `NULL`, puisqu'il n'y a pas de mois avant elle. Les autres montrent la progression mois après mois.

2.4 — Cumul, moyenne mobile et part du total

Théorie : `Theorie_04_SQL.md`, sections 5 et 6.

Objectif : sur ces mêmes ventes mensuelles, ajouter un cumul, une moyenne mobile sur 3 mois et la part du total.

```sql
SELECT annee, mois, ventes,
       SUM(ventes) OVER (ORDER BY annee, mois)                                        AS cumul,
       AVG(ventes) OVER (ORDER BY annee, mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moy_mobile_3,
       ventes / SUM(ventes) OVER ()                                                   AS part_du_total
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
) AS mensuel
ORDER BY annee, mois;
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : 38 mois, et 29 358 677,22 de ventes en tout. La colonne `part_du_total` additionnée donne exactement 1.

Observez. `cumul` monte sans cesse, puisqu'il additionne tout depuis le début ; `moy_mobile_3` lisse les à-coups en moyennant le mois courant et les deux précédents ; `part_du_total` rapporte chaque mois au total de la période, parce que la fenêtre vide, `OVER ()`, couvre tout le résultat. Enlevez le `ROWS BETWEEN …` de la moyenne : elle devient un cumul divisé, ce qui n'est pas ce qu'on veut. Le cadre compte.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_04_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous venez de toucher l'outil analytique le plus puissant du SQL courant. Les requêtes commencent à s'allonger, avec des sous-requêtes imbriquées et des fenêtres répétées : le Cours 05 introduit les CTE (`WITH`) pour les rendre enfin lisibles.

© 2026 iAlexMG. Tous droits réservés.